Primjena SOLID principa u dijagramu klasa

Uvod

U okviru ovog projektnog zadatka razvijen je dijagram klasa koji modelira sistem za upravljanje narudžbama, korisnicima i proizvodima. Prilikom dizajna sistema vodilo se računa o primjeni **SOLID principa**, koji predstavljaju skup smjernica za izgradnju fleksibilnih, proširivih i održivih softverskih sistema.

SOLID principi uključuju:

- Single Responsibility Principle (SRP)
- Open/Closed Principle (OCP)
- Liskov Substitution Principle (LSP)
- Interface Segregation Principle (ISP)
- Dependency Inversion Principle (DIP)

U nastavku je detaljno objašnjeno kako je svaki od ovih principa zadovoljen u prikazanom dijagramu klasa.

1. Single Responsibility Principle (SRP)

(Princip jedne odgovornosti)

Definicija:
Svaka klasa treba imati samo jednu odgovornost, odnosno samo jedan razlog za promjenu.

Primjena u dijagramu:

U sistemu su odgovornosti jasno razdvojene:

- **Korisnik** – upravlja podacima o korisniku (ime, email, uloga) i autentifikacijom (prijava, registracija)
- **Korpa** – upravlja stavkama koje korisnik želi kupiti
- **Narudžba** – predstavlja proces naručivanja i sadrži podatke o ukupnoj cijeni, datumu i statusu
- **StavkaNarudžbe** – predstavlja pojedinačnu stavku unutar narudžbe

Napomena: Ona ima metodu izracunajCijenuStavke(). To je odličan primjer SRP-a jer Narudžba ne računa cijenu svake pojedinačne torte i dekoracije, već to delegira stavki, dok ona samo sabira rezultate.

- **Torta** – sadrži informacije o proizvodu (naziv, ID) i upravlja njenom dostupnošću kroz količinu na stanju
- **Dostava** – zadužena za informacije o isporuci
- **Obavještenje** – bavi se slanjem notifikacija korisnicima

Da bismo ispoštovali sve principe, uvođenjem interfejsa trebali bismo dodati na postojeći dijagram klase:

- **CjenovnikNarudžbe** – izračunava ukupnu cijenu
- **ProcjenaSloženostiNarudžbe** – obrađuje narudžbu

Svaka klasa ima jasno definisanu i ograničenu funkcionalnost, što znači da promjena u jednoj oblasti (npr. način slanja obavještenja) ne utiče na druge dijelove sistema.

Zaključak: SRP je ispunjen jer su odgovornosti razdvojene po klasama.

2. Open/Closed Principle (OCP)

(Otvoreno/zatvoreno načelo)

Definicija:
Klase trebaju biti otvorene za proširenje, ali zatvorene za modifikaciju.

Primjena u dijagramu:

- Mogli bismo koristiti **apstrakcije i interfejse**, npr:
  - IObavještavanje
  - ICjenovnik
  - IProcjenaSloženosti
- Konkretne implementacije bi bile:
  - EmailObavještenje
  - CjenovnikNarudžbe
  - ProcjenaSloženostiNarudžbe

Ako bismo željeli dodati novu funkcionalnost, npr:

- SMS obavještenja → dodali bismo novu klasu SMSObavještenje

- Novi način obračuna cijene → nova implementacija ICjenovnik

Postojeći kod se **ne bi morao mijenjati** ako bismo uveli apstrakcije i interfejse, samo bi se proširio.

Zaključak: Sistem bi podržavao proširenje bez izmjene postojećih klasa → OCP bi bio zadovoljen.

3. Liskov Substitution Principle (LSP)

(Princip zamjenjivosti)

Definicija:
Objekti izvedenih klasa moraju biti zamjenjivi objektima baznih klasa bez narušavanja funkcionalnosti.

Primjena u dijagramu:

- Klase koje bi implementirale interfejse (npr. EmailObavještenje) mogle bi se koristiti svugdje gdje se očekuje IObavještavanje
- Isto važi za:
  - ICjenovnik → CjenovnikNarudžbe
  - IProcjenaSloženosti→ ProcjenaSloženostiNarudžbe

To znači da bi sistem mogao koristiti bilo koju implementaciju bez promjene logike.

Zaključak: Sve implementacije bi poštovale ugovore svojih interfejsa → LSP bi bio zadovoljen.

4. Interface Segregation Principle (ISP)

(Princip segregacije interfejsa)

Definicija:
Klijenti ne trebaju biti prisiljeni implementirati metode koje ne koriste.

Primjena u dijagramu:

Umjesto jednog "velikog" interfejsa, omogućili bismo da sistem koristi više manjih i specijalizovanih interfejsa:

- IObavještavanje → samo slanje poruka
- ICjenovnik → samo obračun cijene

- IProcjenaSloženosti→ samo procesiranje narudžbe

Na ovaj način:

- klase bi implementirale samo ono što im treba
- ne bi bilo nepotrebnih metoda

Zaključak: Interfejsi bi bili mali i fokusirani → ISP bi bio zadovoljen.

5. Dependency Inversion Principle (DIP)

(Princip inverzije zavisnosti)

Definicija:
Visokonivojske klase ne smiju zavisiti od niskonivojskih — obje trebaju zavisiti od apstrakcija.

Primjena u dijagramu:

- U idealnom slučaju, klasa **Narudžba** ne bi trebala zavisiti direktno od konkretnih implementacija:
  - ne bi trebala koristiti direktno EmailObavještenje
  - ne bi trebala koristiti direktno CjenovnikNarudžbe

Umjesto toga bi koristila:

- IObavještavanje
- ICjenovnik
- IProcjenaSloženosti

To bi omogućilo:

- laku zamjenu implementacija
- testiranje (mock objekti)
- fleksibilnost sistema

Zaključak: Zavisnosti bi bile okrenute prema apstrakcijama → DIP bi bio zadovoljen.

Zaključak

Analizom dijagrama može se zaključiti da je SRP u potpunosti ispunjen u trenutnoj verziji, dok su ostali principi (OCP, LSP, ISP, DIP) uzeti u obzir kroz dizajn koji je pripremljen za uvođenje

apstrakcija. Uvođenjem predloženih interfejsa, sistem bi postao maksimalno fleksibilan i modularan:

- SRP → razdvojene odgovornosti
- OCP → sistem otvoren za proširenje
- LSP → pravilna zamjenjivost implementacija
- ISP → mali i specijalizovani interfejsi
- DIP → zavisnosti usmjerene ka apstrakcijama

Ovakav dizajn omogućava:

- lakše održavanje sistema
- jednostavno proširenje funkcionalnosti
- bolju testabilnost i modularnost